

Three-way summary: t1_t2_legacy vs maxeffCons vs maxcons

M1, target HR harm = 1.0, $n = 500$, 500 replicates

2026-08-03

```
data_dir <- "."

arm_files <- c(
  t1_t2_legacy = "fs_t1_t2_m1_h10_knoise0_n500_combined_1_500.rds",
  maxeffCons   = "fs_maxeffCons_fb_mr_m1_h10_knoise0_n500_combined_1_500.rds",
  maxcons      = "fs_maxcons_fb_mr_m1_h10_knoise0_n500_combined_1_500.rds"
)

#' Load simulation bundles and resolve the legacy/new column naming.
#'
#' Legacy runs record the full bootstrap as `t1_*` and multiplier resampling as
#' `t2_*`; the revised codebase records them as `fb_*` and `mr_*`. Detection of
#' `t1_H_est` decides which convention applies.
load_arms <- function(data_dir, files = arm_files) {
  arms <- lapply(names(files), function(nm) {
    bundle <- readRDS(file.path(data_dir, files[[nm]]))
    legacy <- "t1_H_est" %in% names(bundle$results)
    list(
      name      = nm,
      results   = bundle$results,
      truth     = bundle$truth,
      meta      = bundle$meta,
      fb        = if (legacy) "t1" else "fb",
      mr        = if (legacy) "t2" else "mr",
      harm_flag = if (legacy) "t2_harm_flag" else "mr_harm_flag",
      mr_ok     = if (legacy) "gate_ok" else "mr_ok",
      fb_secs   = if (legacy) "t1_secs" else "fb_secs",
      mr_secs   = if (legacy) "t2_secs" else "fit_mr_secs"
    )
  })
  names(arms) <- names(files)
  arms
}
```

```

detected_of <- function(arm) {
  arm$results[arm$results$detected == 1, , drop = FALSE]
}

num <- function(x, digits = 3) formatC(x, format = "f", digits = digits)

#' Evaluate a named list of functions over every arm and bind into a table.
arm_table <- function(arms, rows) {
  body <- lapply(rows, function(f) vapply(arms, f, character(1)))
  df <- as.data.frame(do.call(rbind, body), stringsAsFactors = FALSE)
  cbind(Metric = names(rows), df, row.names = NULL)
}

#' Point estimate, interval length, bias and coverage for one estimator,
#' evaluated against the conditional target  $\beta(\widehat{H})$ .
est_summary <- function(arm, prefix, block = "H") {
  d <- detected_of(arm)
  est <- d[[paste0(prefix, "_", block, "_est")]]
  lo <- d[[paste0(prefix, "_", block, "_lo")]]
  hi <- d[[paste0(prefix, "_", block, "_hi")]]
  tgt <- d[[paste0("betaHhat_", block)]]
  ok <- is.finite(est) & is.finite(lo) & is.finite(hi) & is.finite(tgt)
  c(
    avg = mean(est[ok]),
    len = median((hi - lo)[ok]),
    bias = 100 * mean((est[ok] - tgt[ok]) / tgt[ok]),
    cov = mean(tgt[ok] >= lo[ok] & tgt[ok] <= hi[ok])
  )
}

prefix_for <- function(arm, estimator) {
  switch(estimator, oracle = "or", naive = "nv", FB = arm$fb, MR = arm$mr)
}

estimation_table <- function(arms, block = "H",
                             estimators = c("oracle", "naive", "FB", "MR")) {
  metrics <- c(avg = "Avg", len = "Len", bias = "Bias (\\%)", cov = "Cov")
  parts <- lapply(estimators, function(e) {
    vals <- lapply(arms, function(a) est_summary(a, prefix_for(a, e), block))
    tab <- vapply(names(metrics), function(m) {
      digits <- if (m == "bias") 1 else 3
      vapply(vals, function(v) num(v[[m]], digits), character(1))
    }, character(length(arms)))
    data.frame(
      Estimator = c(e, rep("", length(metrics) - 1L)),
      Metric = unname(metrics),
      t(tab),

```

```

    check.names = FALSE, row.names = NULL, stringsAsFactors = FALSE
  )
})
do.call(rbind, parts)
}

show_table <- function(df, caption) {
  knitr::kable(df, booktabs = TRUE, linesep = "", align = "l",
    escape = FALSE, caption = caption)
}

arms <- load_arms(data_dir)

```

All three bundles share the design cell: `n_sample` 500, `n_sims` 500, `nb_boots` 300, `draws` 5000, `seed_base` 8316951, `consistency_method` = "resample", `forestsearch` 0.2.0, and identical truth. The arms differ only in `sg_focus`: "eff", "maxeffCons", "maxcons".

```

design <- data.frame(
  arm      = names(arms),
  sg_focus = vapply(arms, function(a) {
    if (is.null(a$meta$sg_focus)) "eff (legacy)" else a$meta$sg_focus
  }, character(1)),
  n_sample = vapply(arms, function(a) a$meta$n_sample, integer(1)),
  n_sims    = vapply(arms, function(a) a$meta$n_sims, integer(1)),
  nb_boots  = vapply(arms, function(a) a$meta$nb_boots, integer(1)),
  seed_base = vapply(arms, function(a) a$meta$seed_base, integer(1)),
  batches   = vapply(arms, function(a) a$meta$n_batches, integer(1)),
  row.names = NULL
)
show_table(design, "Design constants and provenance.")

```

Table 1: Design constants and provenance.

arm	sg_focus	n_sample	n_sims	nb_boots	seed_base	batches
t1_t2_legacy	eff (legacy)	500	500	300	8316951	4
maxeffCons	maxeffCons	500	500	300	8316951	3
maxcons	maxcons	500	500	300	8316951	3

```

# truth is identical across arms
stopifnot(
  all.equal(arms$t1_t2_legacy$truth, arms$maxeffCons$truth),
  all.equal(arms$t1_t2_legacy$truth, arms$maxcons$truth)
)

```

1 Detection and subgroup identification

Rows below the detection count are over the detected replicates.

```
rows_detection <- list(
  "Detections / 500"      = function(a) as.character(sum(a$results$detected)),
  "Detection rate (\\%)"  = function(a) num(100 * mean(a$results$detected), 1),
  "mean $\\lvert\\widehat{H}\\rvert$"      = function(a) num(mean(detected_of(a)$n_harm), 1),
  "SD $\\lvert\\widehat{H}\\rvert$"      = function(a) num(sd(detected_of(a)$n_harm), 1),
  "median $\\lvert\\widehat{H}\\rvert$"     = function(a) num(median(detected_of(a)$n_harm), 1),
  "min $\\lvert\\widehat{H}\\rvert$"      = function(a) num(min(detected_of(a)$n_harm), 0),
  "max $\\lvert\\widehat{H}\\rvert$"      = function(a) num(max(detected_of(a)$n_harm), 0),
  "mean $\\lvert\\widehat{H}\\rvert$ (\\% ITT)" = function(a) {
    num(100 * mean(detected_of(a)$n_harm) / a$meta$n_sample, 1)
  },
  "mean true Q"          = function(a) num(mean(detected_of(a)$n_true), 1),
  "Sensitivity"           = function(a) num(mean(detected_of(a)$sens), 1),
  "Specificity"           = function(a) num(mean(detected_of(a)$spec), 1),
  "PPV"                   = function(a) num(mean(detected_of(a)$ppv), 1),
  "NPV"                   = function(a) num(mean(detected_of(a)$npv), 1),
  "MR harm flags"         = function(a) {
    as.character(sum(detected_of(a)[[a$harm_flag]], na.rm = TRUE))
  },
  "MR returned estimate" = function(a) {
    d <- detected_of(a)
    sprintf("%d / %d", sum(d[[a$mr_ok]], na.rm = TRUE), nrow(d))
  },
  "mean $\\beta(\\widehat{H})$"          = function(a) num(mean(detected_of(a)$betaHhat_H))
)

show_table(arm_table(arms, rows_detection),
  "Detection and subgroup identification.")
```

Table 2: Detection and subgroup identification.

Metric	t1_t2_legacy	maxeffCons	maxcons
Detections / 500	346	346	346
Detection rate (%)	69.2	69.2	69.2
mean $ \widehat{H} $	82.3	78.1	87.5
SD $ \widehat{H} $	21.8	17.8	28.1
median $ \widehat{H} $	76	73	79
min $ \widehat{H} $	61	61	61
max $ \widehat{H} $	184	167	220
mean $ \widehat{H} $ (% ITT)	16.5	15.6	17.5
mean true Q	61.9	61.9	61.9
Sensitivity	0.390	0.368	0.405

Table 2: Detection and subgroup identification.

Metric	t1_t2_legacy	maxeffCons	maxcons
Specificity	0.867	0.874	0.858
PPV	0.315	0.309	0.309
NPV	0.909	0.907	0.910
MR harm flags	145	109	134
MR returned estimate	346 / 346	346 / 346	346 / 346
mean $\beta(\widehat{H})$	0.728	0.726	0.725

2 Estimation and coverage — harm block $\widehat{H}$

Avg is the mean point estimate, Len the median interval length; Bias% and Cov are computed against $\beta(\widehat{H})$.

```
show_table(estimation_table(arms, block = "H"),
           "Harm block: estimation and coverage against  $\beta(\widehat{H})$ ."
```

Table 3: Harm block: estimation and coverage against $\beta(\widehat{H})$.

Estimator	Metric	t1_t2_legacy	maxeffCons	maxcons
oracle	Avg	1.135	1.135	1.135
	Len	1.441	1.441	1.441
	Bias (%)	54.9	55.2	55.6
	Cov	0.835	0.838	0.827
naive	Avg	1.629	1.643	1.611
	Len	2.082	2.157	1.964
	Bias (%)	128.0	130.9	125.9
	Cov	0.145	0.153	0.130
FB	Avg	1.222	1.229	1.223
	Len	1.659	1.749	1.648
	Bias (%)	71.0	72.6	71.3
	Cov	0.801	0.806	0.783
MR	Avg	1.003	0.946	0.992
	Len	1.473	1.536	1.395
	Bias (%)	40.0	32.8	38.7
	Cov	0.968	0.986	0.968

The oracle refits on the true region H , so its Bias% and Cov entries are computed against a different region from the one it estimates.

3 Estimation and coverage — complement block $\widehat{H}^c$

```
show_table(estimation_table(arms, block = "Hc"),
  "Complement block: estimation and coverage against  $\beta(\widehat{H}^c)$ ."
```

Table 4: Complement block: estimation and coverage against $\beta(\widehat{H}^c)$.

Estimator	Metric	t1_t2_legacy	maxeffCons	maxcons
oracle	Avg	0.660	0.660	0.660
	Len	0.360	0.360	0.360
	Bias (%)	0.1	-0.1	0.1
	Cov	0.960	0.960	0.960
naive	Avg	0.605	0.609	0.598
	Len	0.335	0.336	0.336
	Bias (%)	-8.2	-7.6	-9.2
	Cov	0.916	0.928	0.908
FB	Avg	0.631	0.636	0.626
	Len	0.490	0.487	0.485
	Bias (%)	-4.2	-3.6	-5.0
	Cov	0.983	0.988	0.986
MR	Avg	0.665	0.655	0.658
	Len	0.672	0.679	0.667
	Bias (%)	0.9	-0.7	-0.2
	Cov	1.000	1.000	1.000

4 Timings

Legacy records `t1_secs` and `t2_secs`; the revised arms record `fb_secs` and `fit_mr_secs`. These are the same measured quantities. FB rows are over detected replicates, fit+MR rows over all 500.

```
rows_timing <- list(
  "FB mean (s)"      = function(a) num(mean(detected_of(a)[[a$fb_secs]], na.rm = TRUE), 1),
  "FB median (s)"    = function(a) num(median(detected_of(a)[[a$fb_secs]], na.rm = TRUE), 1),
  "FB min (s)"       = function(a) num(min(detected_of(a)[[a$fb_secs]], na.rm = TRUE), 1),
  "FB max (s)"       = function(a) num(max(detected_of(a)[[a$fb_secs]], na.rm = TRUE), 1),
  "fit+MR mean (s)"  = function(a) num(mean(a$results[[a$mr_secs]], na.rm = TRUE), 2),
  "fit+MR median (s)" = function(a) num(median(a$results[[a$mr_secs]], na.rm = TRUE), 2),
  "fit+MR min (s)"   = function(a) num(min(a$results[[a$mr_secs]], na.rm = TRUE), 2),
  "fit+MR max (s)"   = function(a) num(max(a$results[[a$mr_secs]], na.rm = TRUE), 2),
  "batches pooled"   = function(a) as.character(a$meta$n_batches)
)

show_table(arm_table(arms, rows_timing),
  "Wall-clock seconds per replicate under 115 workers.")
```

Table 5: Wall-clock seconds per replicate under 115 workers.

Metric	t1_t2_legacy	maxeffCons	maxcons
FB mean (s)	166.4	136.2	167.5
FB median (s)	117.2	135.2	185.4
FB min (s)	100.8	105.2	113.3
FB max (s)	397.6	165.3	192.1
fit+MR mean (s)	12.78	5.01	12.66
fit+MR median (s)	11.38	5.41	10.99
fit+MR min (s)	3.37	2.14	3.33
fit+MR max (s)	23.64	6.97	23.47
batches pooled	4	3	3

Timings vary by batch within each arm, so cross-arm differences reflect render conditions as well as any code effect.

5 Session information

```
sessionInfo()
```

R version 4.6.1 (2026-06-24)

Platform: x86_64-pc-linux-gnu

Running under: Pop!_OS 24.04 LTS

Matrix products: default

BLAS: /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0

LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0 LAPACK version 3.12.0

locale:

```
[1] LC_CTYPE=en_US.UTF-8      LC_NUMERIC=C
[3] LC_TIME=en_US.UTF-8      LC_COLLATE=en_US.UTF-8
[5] LC_MONETARY=en_US.UTF-8  LC_MESSAGES=en_US.UTF-8
[7] LC_PAPER=en_US.UTF-8     LC_NAME=C
[9] LC_ADDRESS=C             LC_TELEPHONE=C
[11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C
```

time zone: America/Los_Angeles

tzcode source: system (glibc)

attached base packages:

```
[1] stats      graphics  grDevices  utils      datasets  methods    base
```

loaded via a namespace (and not attached):

```
[1] compiler_4.6.1  fastmap_1.2.0    cli_3.6.6        tools_4.6.1
```

Table 6

```

#' Mean seconds per replicate within `sim_id` segments. Batch boundaries are
#' not stored numerically in `meta`, so they are supplied from
#' `meta$source_files`.
timing_by_batch <- function(arm, breaks, secs = c("fb", "mr")) {
  secs <- match.arg(secs)
  d <- arm$results
  keep <- if (secs == "fb") d$detected == 1 else rep(TRUE, nrow(d))
  col <- if (secs == "fb") arm$fb_secs else arm$mr_secs
  seg <- cut(d$sim_id, breaks = breaks)
  round(tapply(d[[col]][keep], seg[keep], mean, na.rm = TRUE), 2)
}

batch_breaks <- list(
  t1_t2_legacy = c(0, 20, 100, 200, 500),
  maxeffCons = c(0, 50, 300, 500),
  maxcons = c(0, 20, 300, 500)
)

for (nm in names(arms)) {
  cat("\n", nm, " --- FB: ",
      paste(timing_by_batch(arms[[nm]], batch_breaks[[nm]], "fb"), collapse = " / "),
      " | fit+MR: ",
      paste(timing_by_batch(arms[[nm]], batch_breaks[[nm]], "mr"), collapse = " / "),
      "\n", sep = "")
}

```

```

t1_t2_legacy --- FB: 113.29 / 372.26 / 179.83 / 114.46 | fit+MR: 15.7 / 9.11 / 5.02 / 16.15

maxeffCons --- FB: 132.76 / 131.67 / 142.92 | fit+MR: 4.85 / 4.92 / 5.16

maxcons --- FB: 187 / 187.85 / 136.46 | fit+MR: 16.57 / 17.85 / 5.02

```



```
[5] htmltools_0.5.9    otl_0.2.0          rstudioapi_0.19.0 yamll_2.3.12
[9] rmarkdown_2.31     knitr_1.51         jsonlite_2.0.0    xfun_0.60
[13] digest_0.6.39      rlang_1.3.0        evaluate_1.0.5
```